

I. A COMPARATIVE STUDY OF API DEPLOYMENT ON ON-PREM, EC2, AND AWS LAMBDA - RESULTS AND ANALYSIS

In this section, key performance metrics, including cold start time, latency, connection time, and response times, were captured to evaluate the scalability, efficiency, and overall performance of the Python API across different deployment platforms. These metrics were instrumented to be captured using the JMeter load testing tool.

1) *Cold Start Time*: Cold start time is the delay that occurs when a server or serverless function is invoked for the first time or after a period of inactivity. Cold starts were identified as the first invocation after an idle period, while all subsequent invocations were treated as warm executions and measured separately. Cold start times are influenced by factors such as the chosen runtime, the size of the deployment package, and memory allocation. This metric is important because it affects the responsiveness of latency-sensitive APIs, and minimizing cold starts is critical for maintaining consistent performance.

Transaction Name	On-Prem	Cloud	Serverless
Add items	92ms	90ms	231ms
Get Items	19ms	83ms	55ms
Update Put items	21ms	86ms	148ms
Delete items	22ms	84ms	61ms

TABLE I
COLD START RESPONSE TIMES ACROSS THREE PLATFORMS

Serverless consistently shows the highest cold start times compared to On-Prem and Cloud deployments due to container initialization and stateless execution.

2) *Response Times*: Response time is the total duration from when a client sends a request to a server until the entire response is received. It reflects the complete end-to-end performance of a transaction, providing a comprehensive view of how quickly the system handles and delivers requests.

Transaction Name	On-Prem	Cloud	Serverless
Add items	49ms	148ms	261ms
Get Items	45.2ms	144ms	72.4ms
Update Put items	43ms	140.7ms	99.6ms
Delete items	47.4ms	456ms	149ms

TABLE II
AVERAGE TRANSACTION RESPONSE TIMES ACROSS THREE PLATFORMS

Figure. ?? is the graphical representation of response times for all 4 transactions tested on all three platforms. Google sheets [?] was used to present the results in graphical format. Max value for the y-axis was set at 500ms to better represent the results for all other faster transactions. Response times for the transactions that are slower than 500ms were cropped in the figure.

On-Prem deployment is consistently the fastest due to local hosting and a predictable environment. Cloud deployment shows moderate response times due to network overhead and resource sharing. Serverless response times vary based on cold/warm starts and multiple concurrent instances.

3) *Latency*: Latency is the time elapsed from when a request is sent to the server until the first byte of the response is received. It differs from response time, which measures the total duration until the entire response is received. Latency provides insight into the server's initial responsiveness and network delay, helping to evaluate how quickly the server begins processing and returning data under load.

Transaction Name	On-Prem	Cloud	Serverless
Add items	49.1ms	90.8ms	179ms
Get Items	43.1ms	91.2ms	72ms
Update Put items	43ms	91.2ms	190.7ms
Delete items	47.3ms	90.3ms	99.6ms

TABLE III
LATENCY ACROSS THREE PLATFORMS

Figure 2 is the graphical representation of Latency times for all 4 transactions tested on all three platforms

Serverless APIs introduce higher latency due to their stateless nature and multiple instances, while the Cloud shows slightly higher latency than On-Prem but remains predictable. However, given the maintenance-free nature

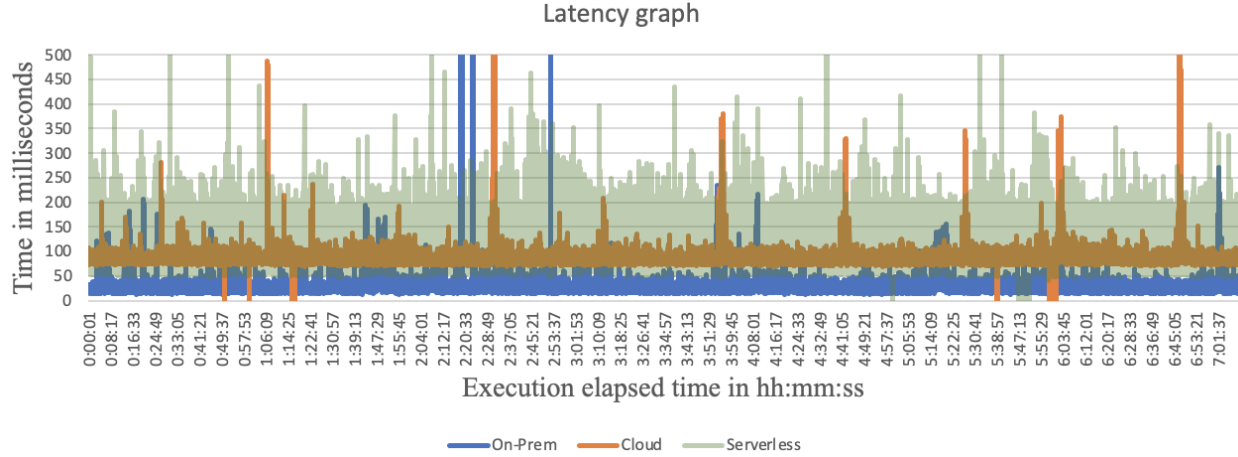


Fig. 2. Latency graph for all transactions tested in all three platforms

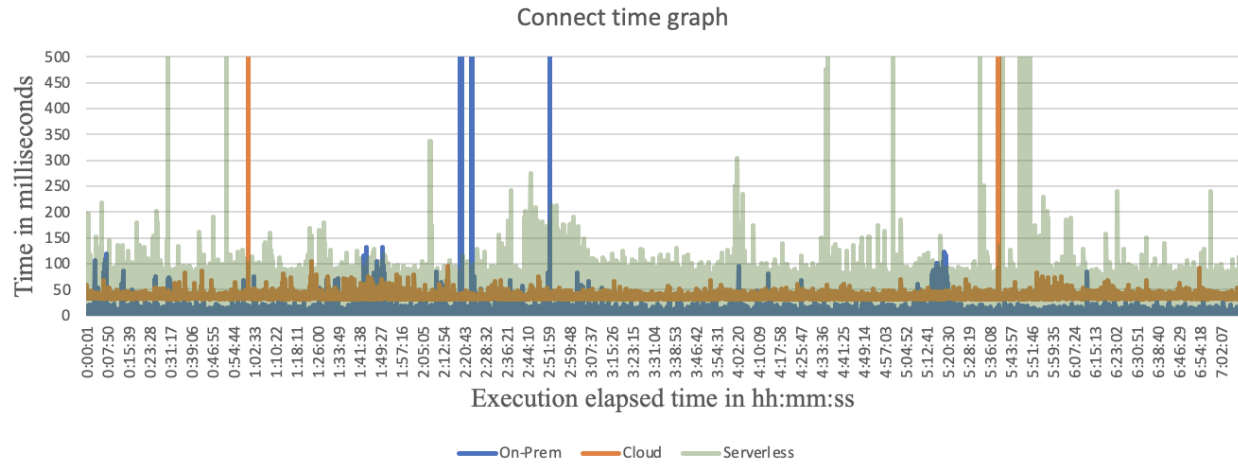


Fig. 3. Connection time graph for all transactions tested in all three platforms

of cloud servers, this small increase in latency is generally acceptable, as it does not significantly impact scalability or operational efficiency, and the benefits of reduced management overhead often outweigh the minor performance trade-off.

4) *Connection Time*: Connection time refers to the time taken to establish a network connection between the client (JMeter) and the server before any request is sent. Connection time is important because it reflects network overhead and server accessibility.

Transaction Name	On-Prem	Cloud	Serverless
Add items	26.5ms	36.3ms	163ms
Get Items	20.2ms	36.4ms	0.1ms
Update Put items	19.9ms	38.9ms	0.13ms
Delete items	25ms	38.9ms	149ms

TABLE IV
CONNECTION TIME ACROSS THREE PLATFORMS

Figure 3 is the graphical representation of the connection time for all 4 transactions tested on all three platforms.

Serverless shows high variability in connection time, especially for certain transactions, because cold starts create new connections, while warm instances reuse connections (hence near-zero times for some transactions). On-Prem and Cloud are more consistent.

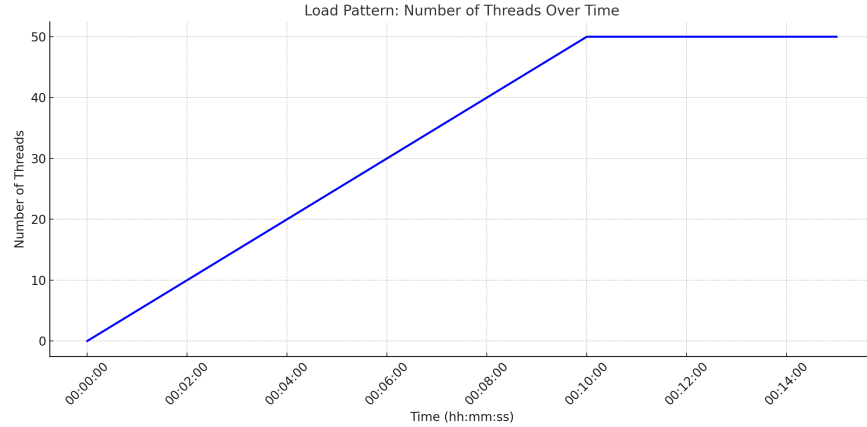


Fig. 4. Number of Sent Requests graph

5) *Scalability*: Scalability refers to a system ability to maintain stability with increasing workloads or demand without compromising the performance of the application.

Figure 4 shows the load pattern simulated using JMeter to evaluate the performance and scalability of the API across all three platforms.

6) *Extended Analysis*: The connect time for the cloud setup is comparable to On-Prem environment, but connect time and latency are slightly deviated from On-Prem. This additional latency is expected, and it is due to the server being hosted in a remote data center. In contrast, the Lambda environment shows noticeably higher connection times, latencies, and ultimately response times when compared to the other two platforms. Client-server geographic placement affects measured latency because network distance and routing between the client and server introduce additional round-trip delay beyond application processing time. With appropriate tuning and optimization, the Lambda implementation has the potential to achieve response times similar to those of the On-Prem and cloud setups. Given its scalability and operational benefits, serverless remains the most advantageous long-term option.